

# Продвинутое машинное обучение

Лекция 4: Gradient Boosting: аддитивность, функциональный градиентный спуск

Автор курса: Лапшина Анастасия (University of Maryland, College Park, ВШЭ)

## Часть 1. Boosting как аддитивная модель

# Аддитивность бустинга

**Bagging** усредняет много деревьев, построенных независимо

**Boosting** добавляет базовые модели последовательно: каждая новая модель исправляет ошибки текущего ансамбля

Итоговая модель - сумма простых базовых моделей

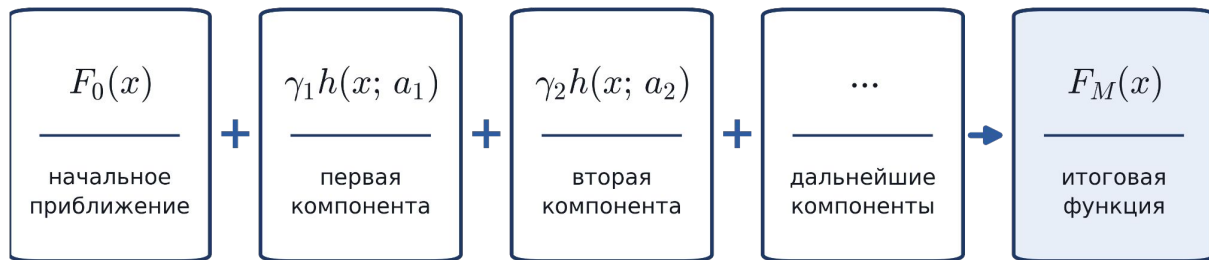
Обучающая выборка:  $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$ , где  $x_i \in \mathbb{R}^p$  - вектор признаков объекта,  $y_i$  - таргет

Базовая модель  $h(x; a_m)$  - простой алгоритм (часто короткое дерево), где  $x$  - произвольный объект,  $a_m$  - параметры  $m$ -ой базовой модели

На шаге  $m$  добавляем новую компоненту с весом  $\gamma_m$

Итоговая функция - аддитивная сумма компонент:

$$F_M(x) = F_0(x) + \sum_{m=1}^M \gamma_m h(x; a_m)$$



# Forward stagewise additive modeling

Как именно бустинг добавляет новую компоненту

1. **Инициализация.** Выбираем константное начальное приближение:

$$c^* \in \arg \min_{c \in \mathbb{R}} \sum_{i=1}^n L(y_i, c), \quad F_0(x) \equiv c^* \quad \forall x$$

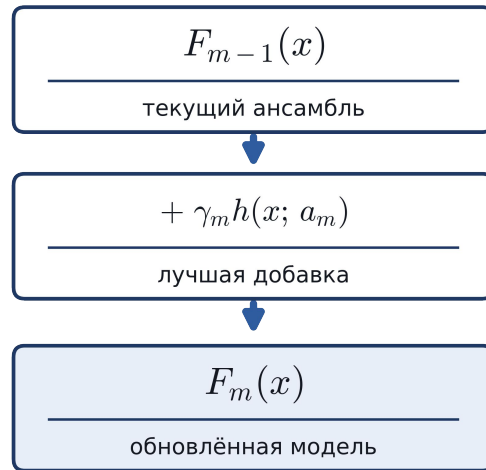
*Например, для MSE:  $c^*$  = среднее значение target на train.*

2. **На шаге  $m$**  ищем лучшую поправку:

$$(\gamma_m, a_m) = \arg \min_{\gamma, a} \sum_{i=1}^n L(y_i, F_{m-1}(x_i) + \gamma h(x_i; a))$$

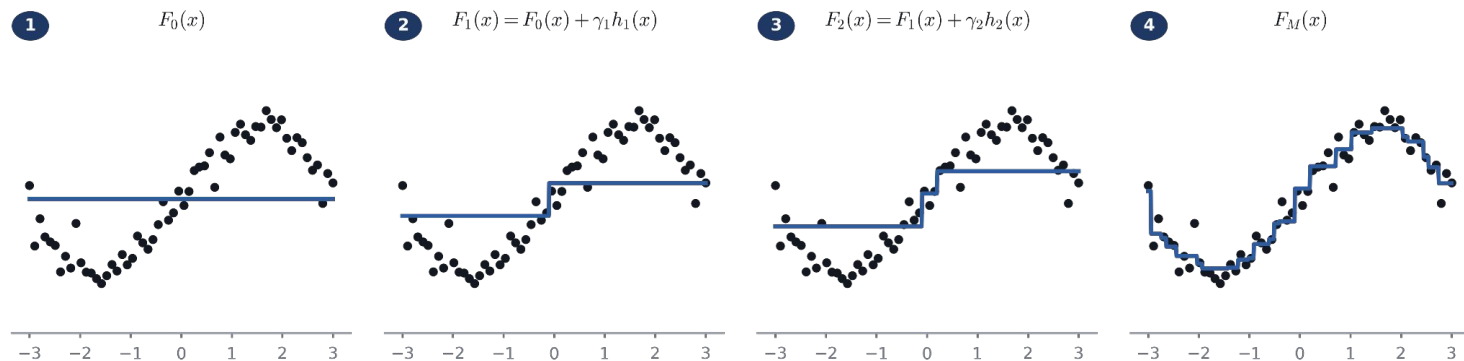
3. **Обновляем ансамбль:**

$$F_m(x) = F_{m-1}(x) + \gamma_m h(x; a_m)$$



Boosting - пошаговая минимизация эмпирического риска в классе аддитивных моделей.

# Интуиция градиентного бустинга



$$F_M(x) = F_0(x) + \gamma_1 h(x; a_1) + \gamma_2 h(x; a_2) + \cdots + \gamma_M h(x; a_M).$$



Ансамбль постепенно улучшает приближение. При этом каждая компонента - базовая модель - маленькая и простая, но полезная

# Алгоритм

1. Считаем псевдо-остатки для всех объектов:

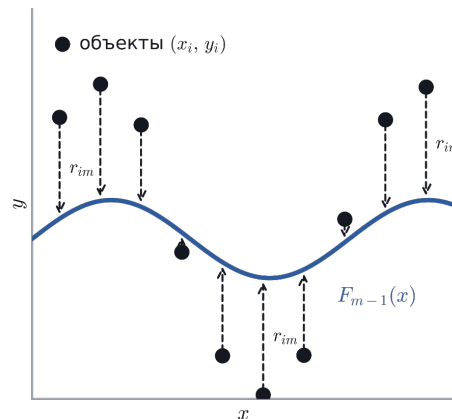
$$r_{im} = - \left[ \frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right]_{F=F_{m-1}}$$

2. Обучаем базовую модель  $h(x; a_m)$  предсказывать эти псевдо-остатки

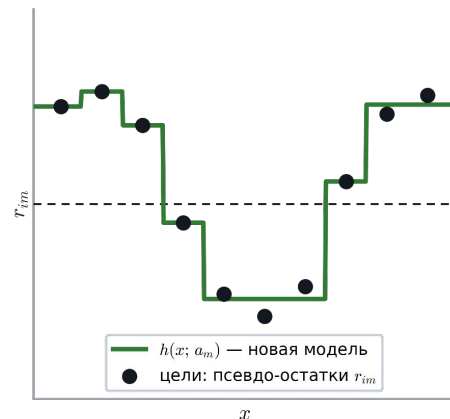
3. Подбираем коэффициент  $\gamma_m$  и обновляем ансамбль:

$$\gamma_m = \arg \min_{\gamma} \sum_{i=1}^n L(y_i, F_{m-1}(x_i) + \gamma h(x_i; a_m))$$

объекты и текущее предсказание



новая модель подгоняет псевдо-остатки



## Часть 2. От обычных остатков к gradient boosting

# От обычных остатков к псевдо-остаткам

Для квадратичной функции потерь:

$$L(y, F) = \frac{1}{2}(y - F)^2$$

$$\frac{\partial L(y, F)}{\partial F} = F - y$$

То есть, для MSE отрицательный градиент совпадает с обычным остатком

## Числовая интуиция

Истинный target  $y_i = 10$

Текущее предсказание  $F_{m-1}(x_i) = 7$

**Псевдо-остаток**  $r_{im} = 10 - 7 = 3$



Новое дерево должно дать  
положительную правку

# Общий случай функции потерь

Эмпирический риск (средний loss):

$$\hat{R}(F) = \frac{1}{n} \sum_{i=1}^n L(y_i, F(x_i))$$

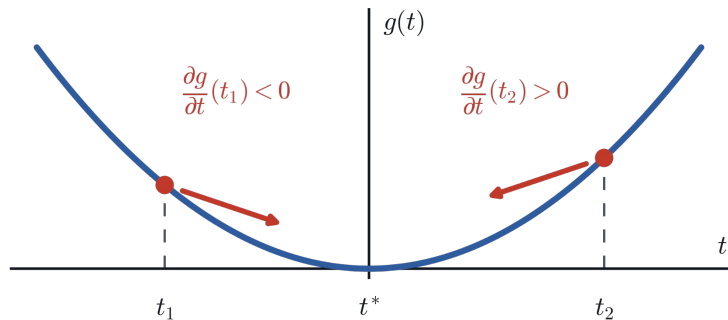
На обучающей выборке текущую функцию можно представить вектором предсказаний:

$$\mathbf{F}_{m-1} = (F_{m-1}(x_1), \dots, F_{m-1}(x_n))^\top$$

Тогда отрицательный градиент риска по этим координатам - это псевдо-остатки:

$$r_{m,i} = - \left. \frac{\partial \hat{R}(F)}{\partial F(x_i)} \right|_{\mathbf{F}=\mathbf{F}_{m-1}} = - \frac{1}{n} \left. \frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right|_{\mathbf{F}=\mathbf{F}_{m-1}}$$

Каждая координата  $r_{m,i}$  отвечает: как локально (для конкретно этого объекта) изменить предсказание, чтобы уменьшить исходный loss



$$\frac{\partial g}{\partial t} < 0$$

двигаемся вправо  
(увеличиваем  $t$ )

$$\frac{\partial g}{\partial t} > 0$$

двигаемся влево  
(уменьшаем  $t$ )

антиградиент направлен в сторону  
**локального уменьшения функции потерь**

~~Антиградиент посчитали – теперь прибавим его к  
исходному вектору функций (как в обычном  
градиентном спуске)~~

Почему так нельзя?

## Часть 3. Функциональный градиентный спуск

# Антиградиент задает направление только для train объектов

На эмпирическом риске функцию  $F$  видим через  $n$  значений - по одному на каждый объект обучающей выборки.

**Функция как вектор предсказаний:**

$$\hat{R}(F) = \frac{1}{n} \sum_{i=1}^n L(y_i, F(x_i))$$

$$\mathbf{F}_{m-1}(X) = (F_{m-1}(x_1), \dots, F_{m-1}(x_n))^{\top}$$

В этих  $n$  координатах риск – обычная дифференцируемая функция (лосс)

**Идеальный локальный шаг:**

$$\mathbf{r}_m = -\nabla_{\mathbf{F}} \hat{R}(F_{m-1})$$

$$\mathbf{F}_m^{\text{ideal}} = \mathbf{F}_{m-1} + \alpha \mathbf{r}_m$$

Такой шаг корректен в  $\mathbb{R}^n$ : каждая координата двигается в сторону локального уменьшения loss

Но  $\mathbf{r}_m$  содержит только  $n$  поправок. Он не говорит, что прибавить в  $x_{\text{new}}$ .  
Бустингу нужно направление-функция, а не ещё один  $n$ -мерный вектор.

# Базовая модель приближает антиградиент

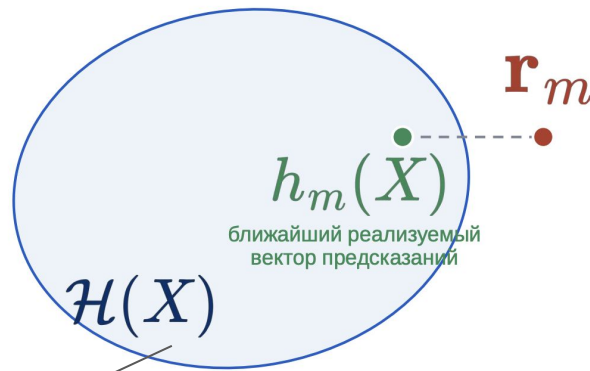
Вместо точного антиградиента берём функцию из класса базовых моделей, чьи train-предсказания ближе всего к  $\mathbf{r}_m$ . Логика такая: если эта базовая модель хорошо приблизила остатки для train-объектов, то наверняка поможет нам и на новых тестовых объектах. Но насколько хорошо она обобщается, проверяется на validation/test

$$h_m \in \arg \min_{h \in \mathcal{H}} \sum_{i=1}^n (r_{m,i} - h(x_i))^2$$

То есть минимизируем расстояние между:

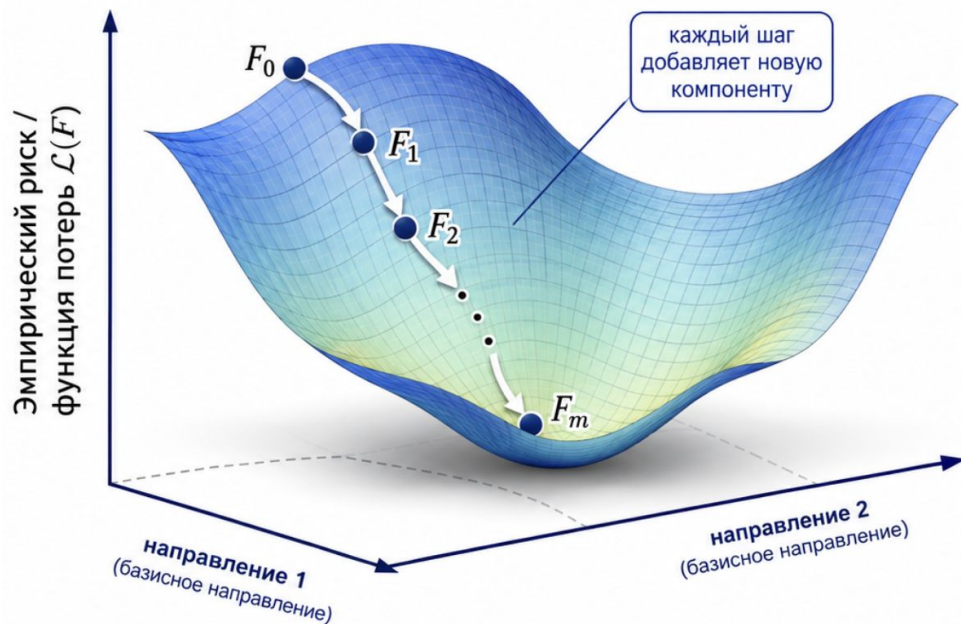
$$\mathbf{r}_m \quad \text{и} \quad h(X) = (h(x_1), \dots, h(x_n))^T$$

- $\mathbf{r}_m$  — идеальное направление в train-координатах
- $h_m(X)$  — направление, реализуемое базовой моделью



Ограниченный класс  $\mathcal{H}$  заставляет модель искать общее правило коррекции, а не запоминать отдельную поправку для каждого train-объекта.

# Спуск в пространстве функций



- 1 Текущая модель:  $F_{m-1}(x)$
- 2 Ищем направление наибольшего убывания функционала:  
 $-\nabla_F \mathcal{L}$
- 3 Базовая модель  $h(x; a_m)$  приближает это направление
- 4 Обновление:  
 $F_m(x) = F_{m-1}(x) + \gamma_m h(x; a_m)$

То есть оптимизируем не параметры отдельной модели, а саму функцию предсказания. Поэтому спуск выполняется **в пространстве функций**, а не в пространстве параметров модели

# Line search

После того как функция, приближающая остатки, зафиксирована, подбираем значение параметра  $\gamma$ , определяющего масштаб вносимой коррекции

$\gamma = 0$

$$F_{m-1}(x) + 0 \cdot h_m(x)$$

модель не добавляем

$\gamma = 0.5$

$$F_{m-1}(x) + 0.5 h_m(x)$$

половина поправки

$\gamma = 1$

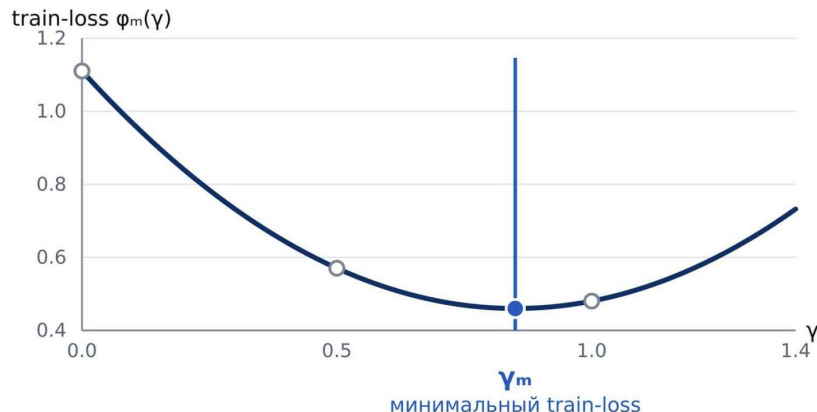
$$F_{m-1}(x) + 1 \cdot h_m(x)$$

полная поправка  $h(x)$

«Масштаб» означает: одно число  $\gamma$  одновременно умножает все значения функции  $h_m(x)$ .

## Выбираем $\gamma$ с минимальным train-loss

$$\phi_m(\gamma) = \hat{R}(F_{m-1} + \gamma h_m), \quad \gamma_m \in \arg \min_{\gamma \in \mathbb{R}} \phi_m(\gamma)$$



$$F_m(x) = F_{m-1}(x) + \eta \gamma_m h_m(x)$$

$\gamma_m$  — подбирается заново на этой итерации  
 $\eta$  — фиксированный learning rate; дополнительно уменьшает шаг

# Что происходит на новых объектах

**Идея:** Псевдо-остатки служат targets только при обучении  $h_m$ ; затем к новым объектам применяется уже выученная функция  $h_m(x)$

ОБУЧЕНИЕ: известны  $x_i$  и  $y_i$

1. Вычисляем псевдо-остатки

$$r_{m,i} = - \left. \frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right|_{F=F_{m-1}}$$

2. Обучаем базовую модель на парах  $(x_i, r_{m,i})$

$$h_m \in \arg \min_{h \in \mathcal{H}} \sum_{i=1}^n (r_{m,i} - h(x_i))^2$$

Результат обучения: функция  $h_m(x)$ ,  
а не таблица из  $n$  отдельных поправок.

НОВЫЙ ОБЪЕКТ: известно только  $x_{\text{new}}$

«Идеальный»  $r_{m,\text{new}}^*$  вычислить нельзя:  
для него потребовался бы неизвестный  $y_{\text{new}}$ .

3. Получаем прогноз коррекции

$$\hat{r}_{m,\text{new}} = h_m(x_{\text{new}})$$

Это оценка, которую  $h_m$  выдаёт по признакам  $x_{\text{new}}$ .

4. Обновляем ансамбль

$$F_m(x_{\text{new}}) = F_{m-1}(x_{\text{new}}) + \eta \gamma_m h_m(x_{\text{new}})$$

На новые объекты переносится функция  $h(x)$ , а не список train-псевдо остатков

## Часть 4. Boosting trees и AdaBoost

# Структура и значения листьев

## 1. Находим области $R_{m,j}$ :

Регрессионное дерево приближает псевдо-остатки методом наименьших квадратов. Так мы группируем train-объекты, которым нужны похожие поправки; результат этого шага - области, а не окончательные значения листьев.

$$\tilde{h}_m \in \arg \min_{h \in \mathcal{T}_J} \sum_{i=1}^n (r_{m,i} - h(x_i))^2$$

Под  $\arg \min$  перебирается класс регрессионных деревьев с  $J$  листьями.  $\tilde{h}_m$  - вспомогательное дерево; здесь нам нужна прежде всего структура разбиения.

## 2. Переоцениваем каждый лист:

После фиксации областей выбираем  $\gamma$ , минимизирующее исходный loss внутри листа. Значит, структура определяется градиентом, а величина коррекции - функцией потерь  $L$ .

$$\gamma_{m,j} \in \arg \min_{\gamma \in \mathbb{R}} \sum_{x_i \in R_{m,j}} L(y_i, F_{m-1}(x_i) + \gamma)$$

$$T_m(x) = \sum_{j=1}^{J_m} \gamma_{m,j} \mathbf{1}\{x \in R_{m,j}\}$$

Для нового  $x$  ровно один индикатор равен 1, поэтому  $T_m(x)$  равно  $\gamma_{m,j}$  выбранного листа.

**Для MSE значение листа – средний текущий остаток объектов в этом листе**

$$\gamma_{m,j} = \frac{1}{|I_{m,j}|} \sum_{i \in I_{m,j}} (y_i - F_{m-1}(x_i)), \quad I_{m,j} = \{i : x_i \in R_{m,j}\}$$

# Почему при MSE значение листа - средний остаток

Внутри одного листа дерево обязано выдать всем объектам одну и ту же поправку  $\gamma$ . MSE выбирает ту константу, которая ближе всего ко всем текущим остаткам.

## Train-объекты в листе $R_{m,2}$

Лист  $R_{m,2}$ : одна область  $\rightarrow$  одно число  $\gamma_{m,2}$

объект 1



$$r_{m,i} = +0.7$$

объект 2



$$r_{m,i} = +1.1$$

объект 3



$$r_{m,i} = +0.9$$

Лучшая общая поправка — центр этих трёх значений:

$$\gamma_{m,2} = \frac{0.7 + 1.1 + 0.9}{3} = 0.9$$

## Почему оптимум - это среднее значение остатков в листе

1. Текущая ошибка train-объекта:  $r_{m,i} = y_i - F_{m-1}(x_i)$

$I_{m,j}$  — индексы train-объектов в листе

2. Ищем одну константу  $\gamma$ , минимизирующую MSE внутри листа:

$$\gamma_{m,j} \in \arg \min_{\gamma \in \mathbb{R}} \sum_{i \in I_{m,j}} (r_{m,i} - \gamma)^2$$

3. Приравниваем производную к нулю:

$$0 = \frac{\partial}{\partial \gamma} \sum_{i \in I_{m,j}} (r_{m,i} - \gamma)^2 = -2 \sum_{i \in I_{m,j}} (r_{m,i} - \gamma) \quad \Rightarrow \quad |I_{m,j}| \gamma = \sum_{i \in I_{m,j}} r_{m,i}$$

$$\gamma_{m,j} = \frac{1}{|I_{m,j}|} \sum_{i \in I_{m,j}} r_{m,i} = \frac{1}{|I_{m,j}|} \sum_{i \in I_{m,j}} (y_i - F_{m-1}(x_i))$$

На inference остаток нового объекта не вычисляем: используем поправку, выученную на train

$$x_{\text{new}} \in R_{m,j} \quad \Rightarrow \quad T_m(x_{\text{new}}) = \gamma_{m,j} = 0.9$$

$$F_m(x_{\text{new}}) = F_{m-1}(x_{\text{new}}) + \eta \cdot 0.9$$

# GB над деревьями

1. Инициализация:

$$c^* \in \arg \min_{c \in \mathbb{R}} \sum_{i=1}^n L(y_i, c), \quad F_0(x) \equiv c^*$$

Для  $m = 1, \dots, M$ :

2. Псевдо-остатки:

$$r_{m,i} = - \left. \frac{\partial \hat{R}}{\partial F(x_i)} \right|_{F=F_{m-1}} = - \frac{1}{n} \cdot \left. \frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right|_{F=F_{m-1}}$$

3. Структура дерева:

Фитим regression tree к парам  $(x_i, r_{m,i})$  : дерево создает области  $R_{m,1}, \dots, R_{m,J_m}$  ,  
объединяя объекты с похожими требуемыми поправками

$$\tilde{h}_m \in \arg \min_{h \in \mathcal{H}} \sum_{i=1}^n (r_{m,i} - h(x_i))^2 \quad R_{m,1}, \dots, R_{m,J_m} \text{ — листья } \tilde{h}_m$$

4. Значения листьев:

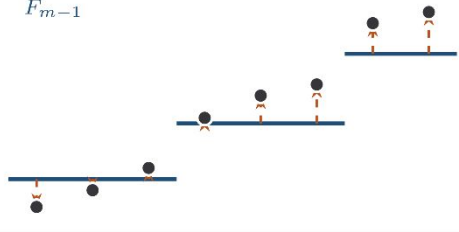
$$\gamma_{m,j} \in \arg \min_{\gamma \in \mathbb{R}} \sum_{x_i \in R_{m,j}} L(y_i, F_{m-1}(x_i) + \gamma)$$

5. Обновление и предсказание:

$$F_m(x) = F_{m-1}(x) + \eta \sum_{j=1}^{J_m} \gamma_{m,j} \mathbf{1}\{x \in R_{m,j}\} \quad \longrightarrow \quad F_M(x) = c^* + \eta \sum_{m=1}^M \sum_{j=1}^{J_m} \gamma_{m,j} \mathbf{1}\{x \in R_{m,j}\}$$

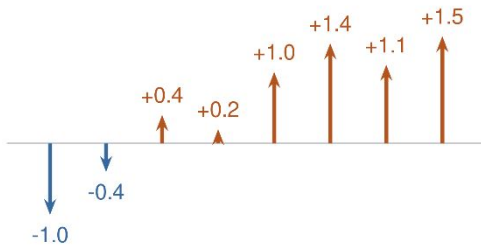
## 1 Где мы сейчас

$F_{m-1}$



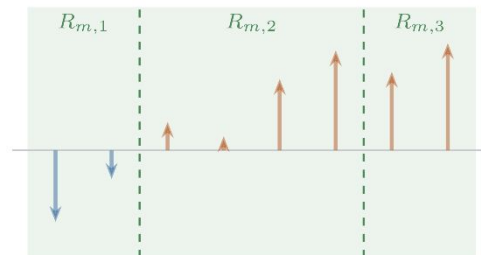
точки —  $y_i$ ,  
ступенька — текущий  $F_{m-1}$

## 2 Псевдо-остатки



для MSE это остаток  
 $y_i - F_{m-1}(x_i)$

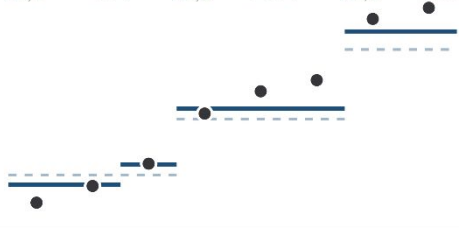
## 3 Дерево группирует



берём только разбиение,  
значения листьев отброшены

## 4 Листья и обновление

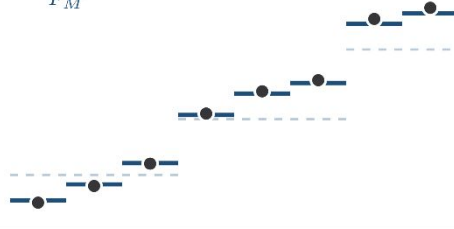
$\gamma_{m,1} = -0.70$     $\gamma_{m,2} = +0.75$     $\gamma_{m,3} = +1.30$



пунктир — было,  
сплошная — стало

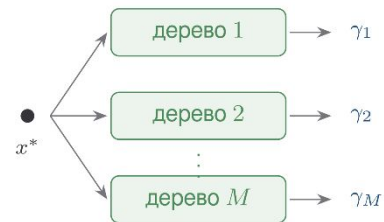
## 5 После $M$ деревьев

$F_M$



ступенька дробится  
и приближается к данным

## 6 Применение



$$F_M(x^*) = c^* + \eta (\gamma_1 + \gamma_2 + \dots + \gamma_M)$$

объект спускается по каждому дереву;  
 $y$  не нужен, остатки не считаются

# AdaBoost: ошибки получают больший вес

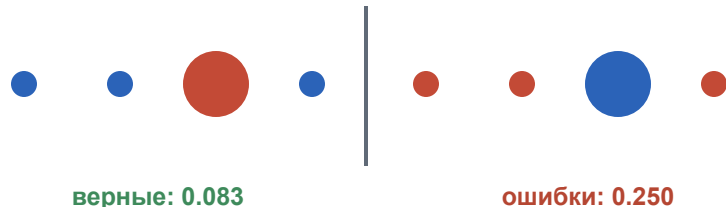
Один раунд на восьми объектах

Цвет = истинный класс  $y_i$ ; размер = вес  $w_i$

До первого дерева:  $w_i = 1/8 = 0.125$



После обновления весов



Численный смысл одного раунда

## 1 Взвешенная ошибка дерева

$$\text{err}_1 = 2 \cdot 0.125 = 0.25$$

## 2 Вес голоса первого дерева

$$\alpha_1 = \frac{1}{2} \log(0.75 / 0.25) \approx 0.55$$

## 3 Нормируем обновлённые веса

верный объект:  $0.125 \rightarrow 0.083$

ошибка:  $0.125 \rightarrow 0.250$

**Ошибка получает в 3 раза больший вес.**

Следующее дерево видит новые веса и сильнее старается исправить текущие ошибки ансамбля.

# AdaBoost: ошибка задаёт голос дерева и новые веса

Бинарная классификация:  $y_i, G(x) \in \{-1, +1\}$

## 0 Инициализация

$$w_i^{(1)} = 1/n, \quad i = 1, \dots, n$$

## 1 Обучаем слабый классификатор $G$

Обычно – stump (пенек – дерево глубины 1). Ошибки объектов учитываются с текущими весами.

$$G_m \in \arg \min_{G \in \mathcal{H}} \sum_{i=1}^n w_i^{(m)} \mathbf{1}\{y_i \neq G(x_i)\}$$

## 2 Считаем взвешенную ошибку

$$\text{err}_m = \sum_{i=1}^n w_i^{(m)} \mathbf{1}\{y_i \neq G_m(x_i)\}, \quad \sum_{i=1}^n w_i^{(m)} = 1$$

Индикатор равен 1 только для ошибочно классифицированных объектов.

Это ансамбль:  $G_2$  обучается на весах после  $G_1$ ,  $G_3$  – после  $G_1$  и  $G_2$ , а прогноз суммирует все деревья.

## 3 Назначаем дереву вес $\alpha$

$$\alpha_m = \frac{1}{2} \log \left( \frac{1 - \text{err}_m}{\text{err}_m} \right)$$

$\text{err}_m < 0.5 \Rightarrow \alpha_m > 0$ . Чем точнее дерево, тем сильнее его голос.

## 4 Обновляем веса объектов

$$w_i^{(m+1)} = \frac{w_i^{(m)} \exp[-\alpha_m y_i G_m(x_i)]}{Z_m}$$

верно:  $y_i G_m(x_i) = +1 \Rightarrow \times e^{-\alpha_m}$   
ошибка:  $y_i G_m(x_i) = -1 \Rightarrow \times e^{+\alpha_m}$

## 5 После $M$ раундов: взвешенное голосование

$$F_M(x) = \sum_{m=1}^M \alpha_m G_m(x), \quad \hat{y}(x) = \text{sign } F_M(x)$$

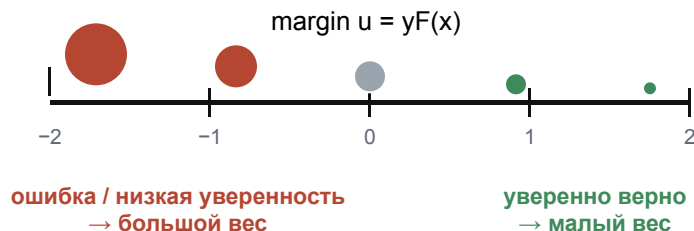
# AdaBoost - stagewise минимизация exponential loss

## Откуда берётся перевзвешивание

$$L_{\text{exp}}(y, F) = \exp(-yF), \quad F_m(x) = F_{m-1}(x) + \alpha_m G_m(x)$$

На каждом шаге выбираем  $G_m$  и  $\alpha_m$ , уменьшающие текущую сумму потерь. Объекты с большой потерей автоматически получают большой вес.

$$u_i = y_i F_{m-1}(x_i), \quad w_i^{(m)} \propto \exp(-u_i)$$



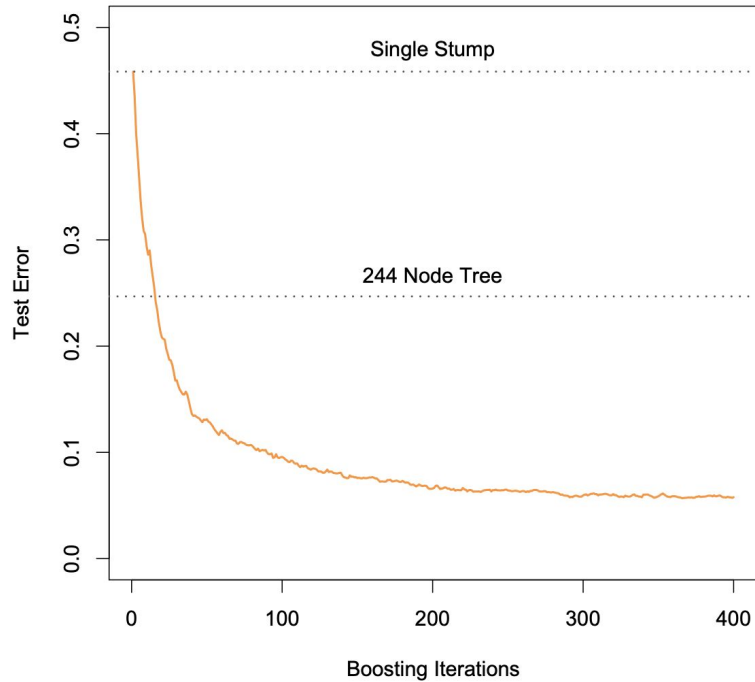
## Сравним с GBT

|                | AdaBoost                  | Gradient boosting trees                     |
|----------------|---------------------------|---------------------------------------------|
| Loss           | exponential               | любой дифференцируемый L                    |
| Сигнал ошибки  | явные веса $w_i$          | псевдо-остатки $r_{m,i}$                    |
| Базовая модель | classifier / stump $G_m$  | regression tree $T_m$                       |
| Величина шага  | одно $\alpha_m$ на дерево | $\gamma_m$ , $\eta$ в каждом листе + $\eta$ |

Цена exponential loss: объекты с большой отрицательной margin получают экспоненциально большой вес. Поэтому AdaBoost чувствителен к label noise и выбросам.

## Часть 5. Перфоманс бустинга

# Stump vs. Tree vs. Boosting



# ИСТОЧНИКИ

1. Hastie, T., Tibshirani, R., Friedman, J. (2009). *The Elements of Statistical Learning*. Springer, Chapter 10.
2. Friedman, J. H. (2001). "Greedy Function Approximation: A Gradient Boosting Machine." *The Annals of Statistics*.
3. Friedman, J., Hastie, T., Tibshirani, R. (2000). "Additive Logistic Regression: A Statistical View of Boosting." *The Annals of Statistics*.
4. Freund, Y., Schapire, R. E. (1997). "A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting." *Journal of Computer and System Sciences*.
5. Breiman, L., Friedman, J. H., Olshen, R. A., Stone, C. J. (1984). *Classification and Regression Trees*. Wadsworth.
6. Mason, L., Baxter, J., Bartlett, P. L., Frean, M. R. (2000). "Boosting Algorithms as Gradient Descent." *NeurIPS 12*.
7. Schapire, R. E., Freund, Y., Bartlett, P., Lee, W. S. (1998). "Boosting the Margin." *The Annals of Statistics*.
8. Breiman, L. (1996). "Bagging Predictors." *Machine Learning*.